



PYTHON PROGRAMMING CONCEPTS

GET STARTED

**PYTHON PROGRAMMING LANGUAGE - YOUR GATEWAY TO MODERN
DEVELOPMENT**



Objective of Session

What is a programming language?

What is program implementation?

What is IDEL and IDE?

What is Compiler and Interpreter ?

What is Virtual Machines (VMS) Python Virtual Machine (PVM) ?

What is source code ?

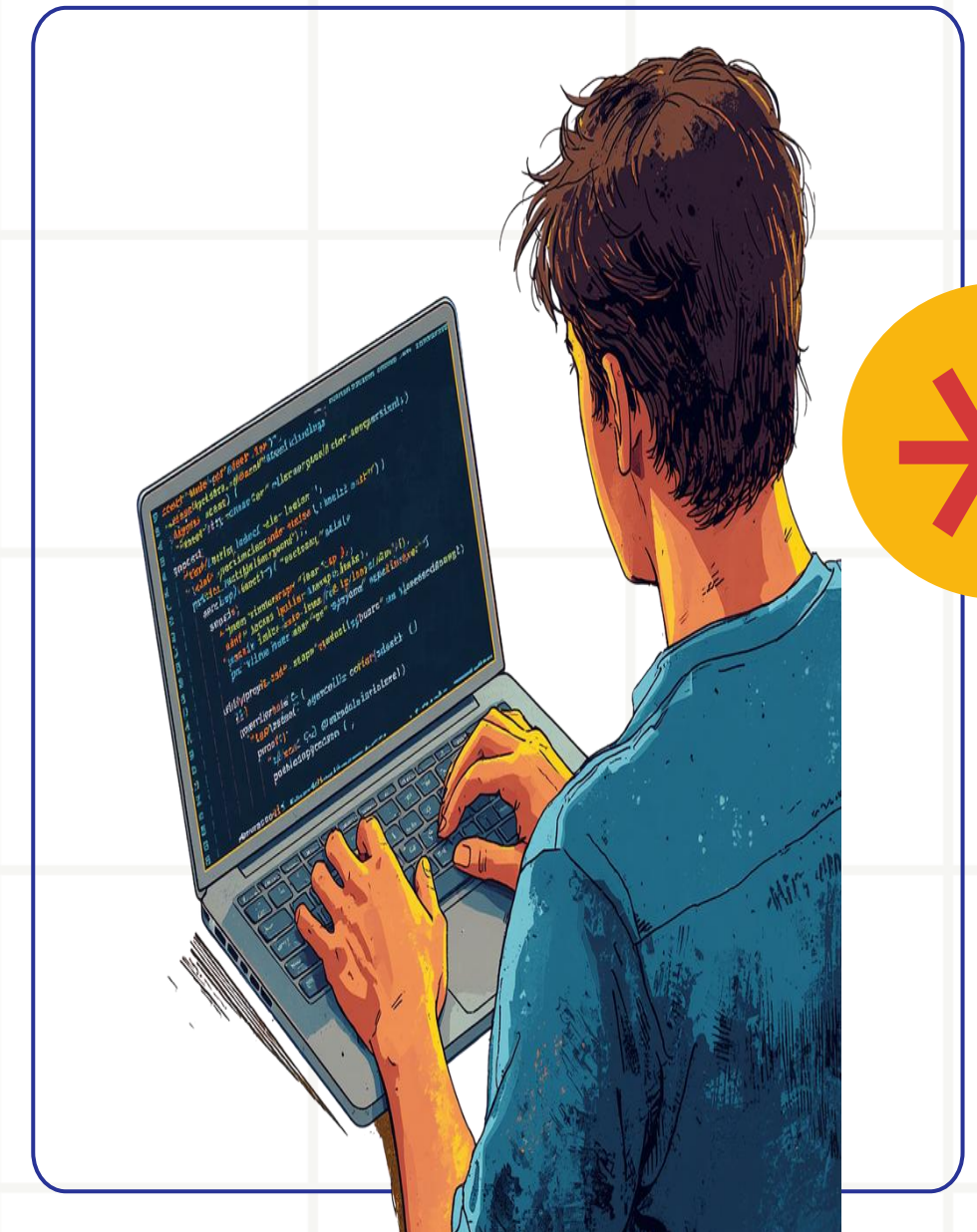
What is compilation ?

What is machine code ?

What is bytecode ?

Summary.

NEXT SLIDE

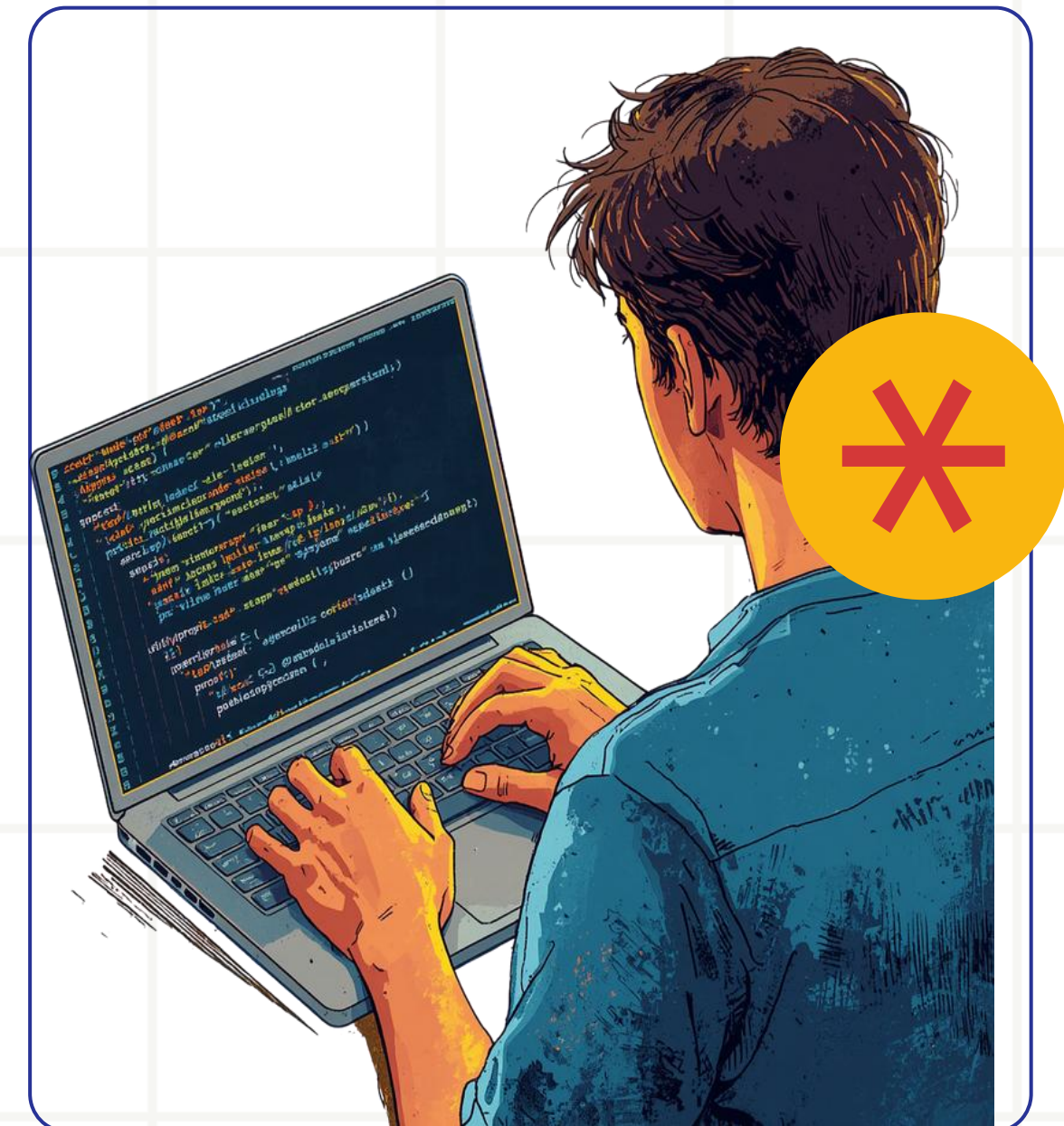




What is a programming language?

A programming language is a formal, structured set of instructions used to communicate with computers. It provides the vocabulary (keywords) and grammar (syntax) that programmers use to write code. Just like human languages have rules, programming languages have strict syntax that must be followed.

Popular examples include Python, C, Java, and JavaScript. Each language has its own strengths and use cases. Python is known for readability, C for system programming, Java for enterprise applications, and JavaScript for web development.

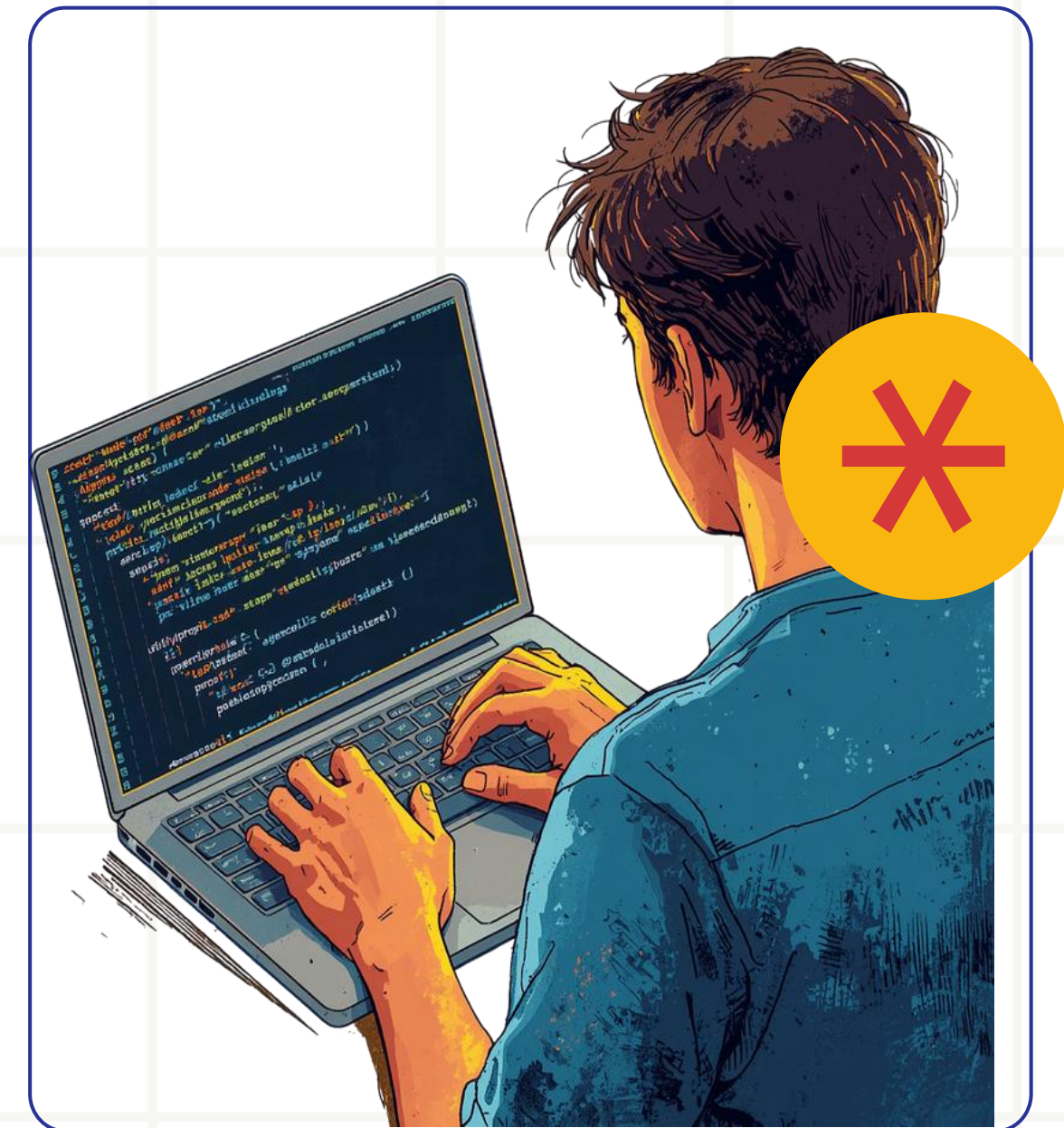
[NEXT SLIDE](#)



Program Implementation

Program implementation is the process of transforming human-readable source code into executable instructions that a computer can understand and run. This critical step bridges the gap between what programmers write and what machines execute.

Three main methods exist: compilers translate entire programs before execution, interpreters process code line-by-line at runtime, and hybrid approaches combine both. The flow is simple: source code → compiler/interpreter → machine execution.

[NEXT SLIDE](#)

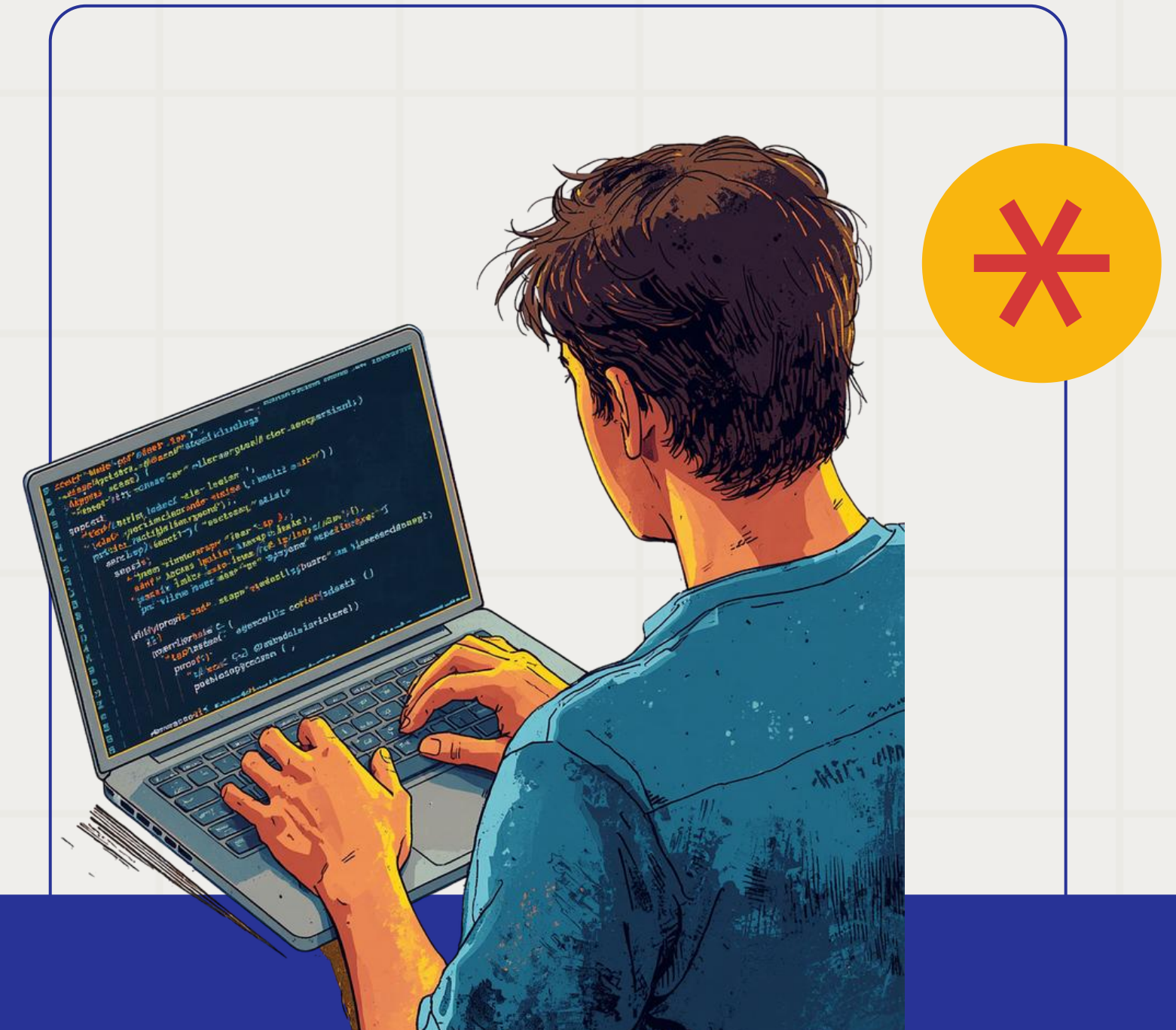
IDLE

Python's built-in editor designed for beginners. Simple interface with basic features like syntax highlighting and interactive shell for quick testing.

IDE

Powerful environments like VS Code, PyCharm, and Thonny. Advanced features include debugging tools, file management, and code completion.

NEXT SLIDE





Compiler vs Interpreter

01. Compiler

Translates entire source code to machine code before execution. Creates standalone executable files. Example: C language. Faster runtime but requires full compilation step.



02. Interpreter

Executes code line-by-line at runtime. No separate compilation step needed. Python's primary method.

Note: Python compiles to bytecode internally despite being interpreted.

NEXT SLIDE



Virtual Machines (VMS) Explained

A Virtual Machine (VM) is a software abstraction that emulates physical computer hardware. It creates a complete computing environment that behaves like a real machine, allowing programs to run as if they were on dedicated hardware.

VMS enable remarkable code portability across different operating systems and hardware platforms. The same code can run on Windows, macOS, or Linux without modification. Examples include Java's JVM and cloud computing instances.



NEXT SLIDE



The Python Virtual Machine (PVM)

The PVM is Python's runtime engine that executes compiled bytecode (.pyc files). When you run a Python program, your source code is first compiled into platform-independent bytecode, which is then loaded into the PVM for execution.

The PVM interprets and executes bytecode instructions line-by-line, acting as the core component that actually runs your Python programs. This abstraction layer enables Python code to run on any system with a compatible PVM installed.



NEXT SLIDE



Machine Code vs Bytecode

Machine Code: Binary instructions (0s and 1s) that the CPU executes directly. Hardware-specific and tied to processor architecture. Fastest execution speed but not portable across different systems.

Bytecode: Intermediate, platform-independent code that Python compiles to before PVM execution. Portable across operating systems and hardware. Enables write once, run anywhere flexibility.



NEXT SLIDE



Types of Python Implementations

CPython: The default and most widely used Python implementation. Written in C, it serves as the reference standard.

PyPy: Features Just-In-Time (JIT) compilation for significantly faster execution.

Jython: Runs on the Java Virtual Machine (JVM), enabling seamless integration with Java libraries.

IronPython: Runs on the .NET framework (CLR) and integrates with .NET libraries.

MicroPython: Lean implementation optimized for microcontrollers and embedded systems with limited memory.

Stackless Python: Enhanced CPython variant supporting microthreads for massive concurrency without threading overhead.



NEXT SLIDE



Python Program Execution Process

Step 1 — Source Code: Write your Python code in a .py file using any text editor or IDE. This human-readable code contains your program logic and instructions.

Step 2 — Compilation: Python compiles source code to bytecode (.pyc files stored in `__pycache__`). The PVM then interprets and executes instructions line-by-line, producing your output.

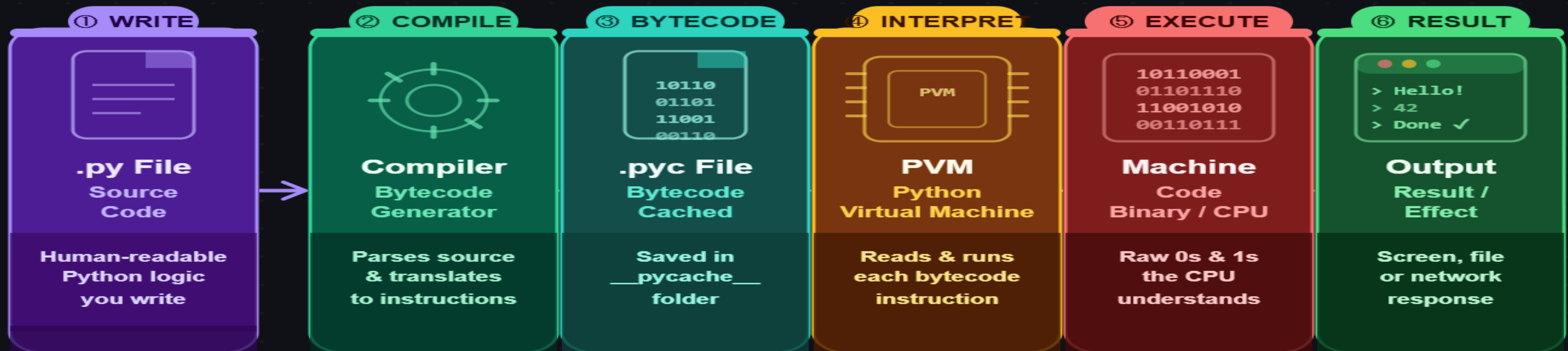
Flow: .py File → Compiler → .pyc Bytecode → PVM → Machine Code → Result.



NEXT SLIDE

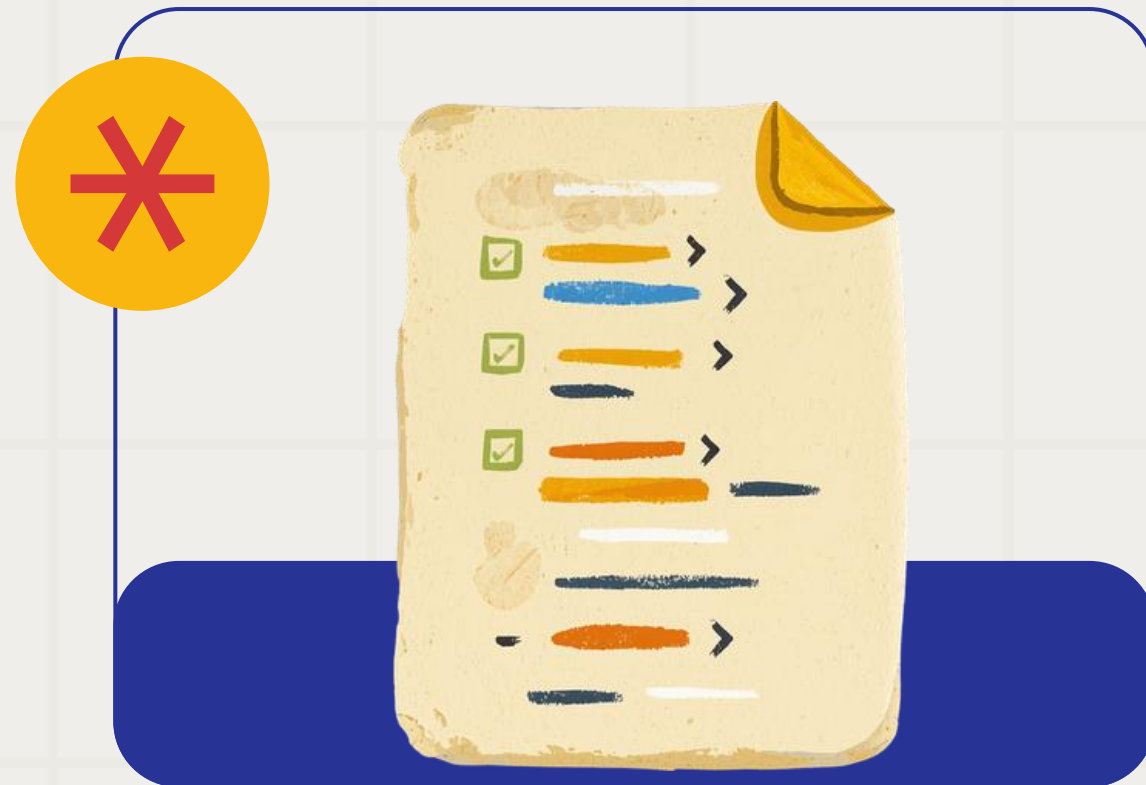
Python Program Execution Process

PYTHON PROGRAM EXECUTION PIPELINE



● Write ● Compile ● Bytecode ● Runtime ● CPU level ● Output

CPython - Bytecode Compiler → PVM → Machine Code → Result



WHY IS PYTHON POPULAR?

Python's popularity stems from its simple, readable syntax that beginners love. Its vast standard library and active community provide endless resources. Plus, it's versatile across web, AI, data science, and more.

NEXT SLIDE



SUMMARY

We covered programming language concepts, program implementation, IDLE vs IDE, compiler vs interpreter, virtual machines, the Python Virtual Machine (PVM), machine code vs bytecode, types of Python implementations, and the Python program execution pipeline.

NEXT SLIDE





QUESTIONS?

Thank you for exploring Python Programming Concepts with us! Feel free to ask any questions or reach out for further information about Python programming.

